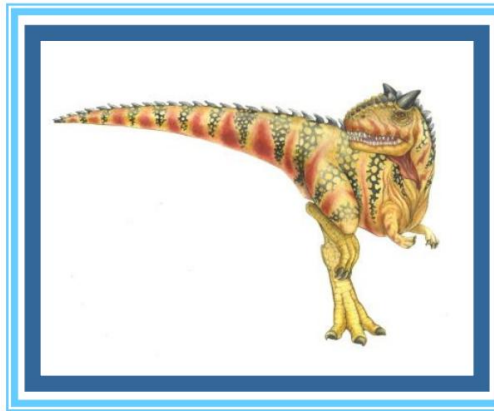


الفصل الثاني: نظام التشغيل الهياكل





الفصل الثاني: هياكل أنظمة التشغيل

- خدمات نظام التشغيل
- واجهة نظام التشغيل للمستخدم
- مكالمات النظام
- أنواع نداءات النظام
- برامج النظام
- تصميم وتنفيذ نظام التشغيل
- بنية نظام التشغيل
- تصحيح أخطاء نظام التشغيل
- إنشاء نظام التشغيل
- تمهيد النظام

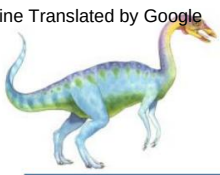




أهداف

- لوصف الخدمات التي يقدمها نظام التشغيل للمستخدمين والعمليات والأنظمة الأخرى
- لمناقشة الطرق المختلفة لبناء نظام التشغيل
- لشرح كيفية تثبيت أنظمة التشغيل وتخصيصها وكيفية تشغيلها





خدمات نظام التشغيل

- توفر أنظمة التشغيل بيئة لتنفيذ البرامج والخدمات للبرامج والمستخدمين
- توفر مجموعة واحدة من خدمات نظام التشغيل وظائف مفيدة للمستخدم:
- واجهة المستخدم -تحتوي جميع أنظمة التشغيل تقريبًا على واجهة مستخدم (UI).
- يختلف بين سطر الأوامر (CLI) ومستخدم الرسومات الواجهة، (GUI)الدفعة
- تنفيذ البرنامج -يجب أن يكون النظام قادرًا على تحميل برنامج في الذاكرة وتشغيل هذا البرنامج، وإنهاء التنفيذ، إما بشكل طبيعي أو غير طبيعي (يشير إلى الخطأ)
- عمليات الإدخال/الإخراج -قد يتطلب البرنامج قيد التشغيل عمليات إدخال/إخراج، والتي قد تتضمن ملفًا أو جهاز إدخال/إخراج





خدمات نظام التشغيل (تابع)



توفر إحدى مجموعات خدمات نظام التشغيل وظائف مفيدة للمستخدم (تابع):



التلاعب بنظام الملفات -نظام الملفات له أهمية خاصة.
تحتاج البرامج إلى قراءة وكتابة الملفات والدلائل، وإنشاءها وحذفها، والبحث عنها، وقائمة معلومات الملف، وإدارة الأذونات.



الاتصالات -قد تتبادل العمليات المعلومات على نفس الكمبيوتر أو بين أجهزة الكمبيوتر عبر الشبكة

□ قد تتم الاتصالات عبر الذاكرة المشتركة أو من خلال الرسائل
المروور (الحزم التي ينقلها نظام التشغيل)

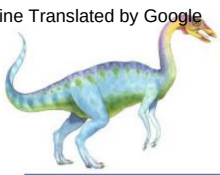


اكتشاف الأخطاء -يجب أن يكون نظام التشغيل على دراية مستمرة بالأخطاء المحتملة
□ قد يحدث في وحدة المعالجة المركزية وأجهزة الذاكرة، وفي أجهزة الإدخال/الإخراج، وفي جهاز المستخدم
برنامج

□ بالنسبة لكل نوع من الأخطاء، يجب على نظام التشغيل اتخاذ الإجراء المناسب لضمان الحوسبة الصحيحة
والمترسقة

□ يمكن أن تعمل مرافق التصحيح على تعزيز قدرات المستخدم والمبرمج بشكل كبير على استخدام النظام
بكفاءة





خدمات نظام التشغيل (تابع)



توجد مجموعة أخرى من وظائف نظام التشغيل لضمان التشغيل الفعال للنظام نفسه من خلال مشاركة الموارد



تخصيص الموارد - عند تشغيل عدة مستخدمين أو عدة وظائف في وقت واحد، يجب تخصيص الموارد لكل منهم

العديد من أنواع الموارد - دورات وحدة المعالجة المركزية، والذاكرة الرئيسية، وتخزين الملفات، أجهزة الإدخال/الإخراج.



المحاسبة - لتتبع المستخدمين الذين يستخدمون موارد الكمبيوتر وكميتها وأنواعها



الحماية والأمان - قد يرغب مالكو المعلومات المخزنة في نظام كمبيوتر متعدد المستخدمين أو متصل بالشبكة في التحكم في استخدام تلك المعلومات، ويجب ألا تتداخل العمليات المتزامنة مع بعضها البعض

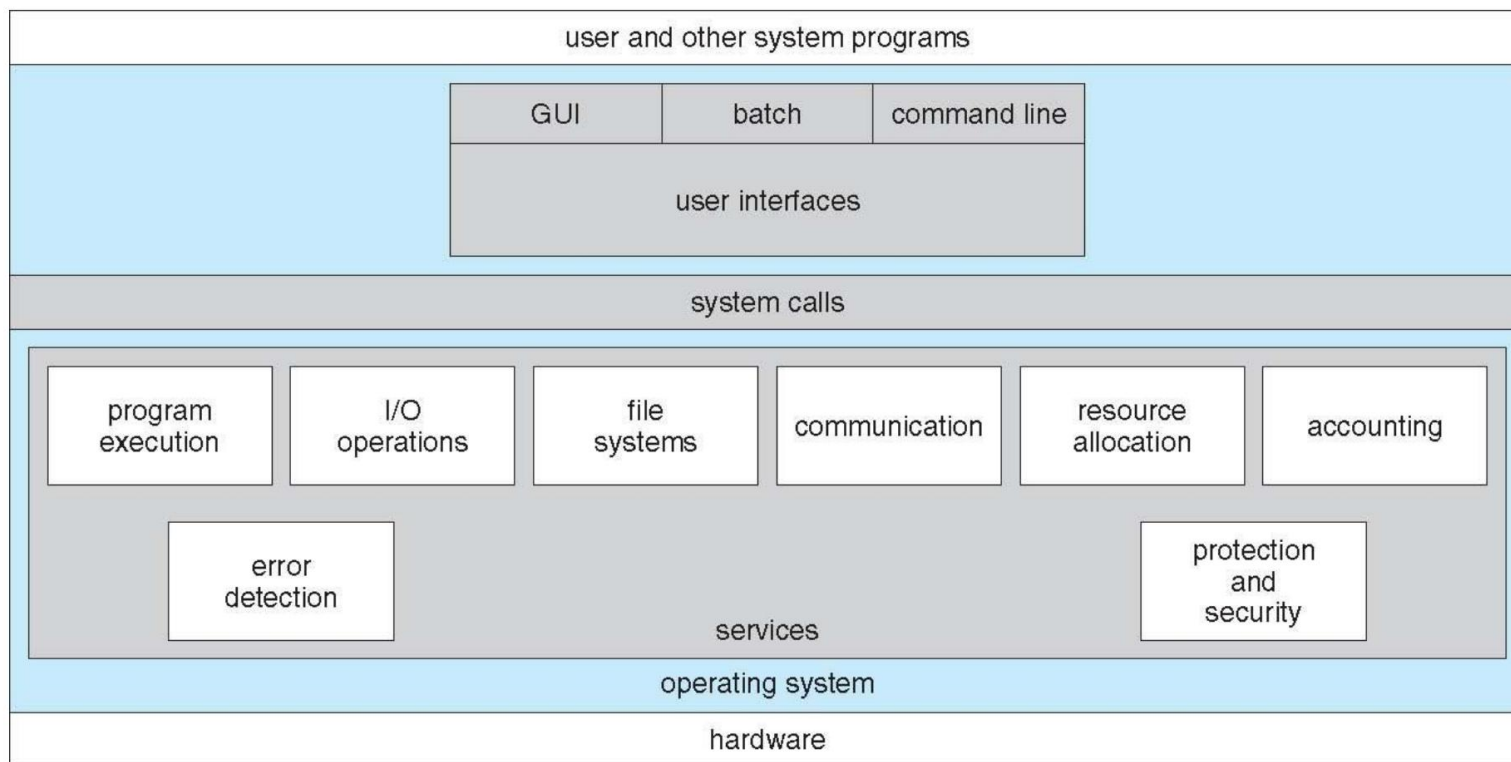
تتضمن الحماية التأكد من أن جميع عمليات الوصول إلى موارد النظام متاحة مُسيطر عليه

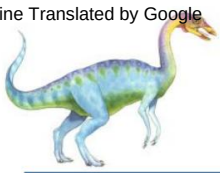
يتطلب تأمين النظام من الغرباء مصادقة المستخدم، ويمتد إلى حماية أجهزة الإدخال/الإخراج الخارجية من محاولات الوصول غير الصالحة





نظرة على خدمات نظام التشغيل





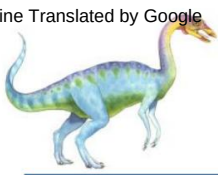
واجهة نظام التشغيل للمستخدم CLI -

يسمح CLI أو **مفسر الأوامر** بإدخال الأوامر بشكل مباشر

- يتم تنفيذه أحياناً في النواة، وأحياناً أخرى بواسطة برنامج النظام
- في بعض الأحيان يتم تطبيق نكهات متعددة -**الأصداف**
- يقوم في المقام الأول بجلب الأمر من المستخدم وتنفيذه
- في بعض الأحيان تكون الأوامر مدمجة، وفي بعض الأحيان تكون مجرد أسماء للبرامج

□ إذا كان الأخير، فإن إضافة ميزات جديدة لا تتطلب تعديل الغلاف





مُفسّر أوامر بورن شل

```

1. root@r6181-d5-us01:~ (ssh)
× root@r6181-d5-u...  × ssh  × root@r6181-d5-us01...  × root@r6181-d5-us01...

Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root    50G       19G   28G  41% /
tmpfs                      127G      520K  127G   1% /dev/shm
/dev/sda1                   477M       71M  381M  16% /boot
/dev/dssd0000               1.0T     480G   545G  47% /dssd_xfs
tcp://192.168.150.1:3334/orangefs 12T     5.7T   6.4T  47% /mnt/orangefs
/dev/gpfs-test              23T     1.1T   22T   5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root      97653 11.2   6.6 42665344 17520636 ?    S<Ll  Jul13 166:23 /usr/lpp/mmfs/bin/mmfsd
root      69849   6.6   0.0      0      0 ?    S     Jul12 181:54 [vpthread-1-1]
root      69850   6.4   0.0      0      0 ?    S     Jul12 177:42 [vpthread-1-2]
root       3829   3.0   0.0      0      0 ?    S     Jun27 730:04 [rp_thread 7:0]
root       3826   3.0   0.0      0      0 ?    S     Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfsd
-r-x----- 1 root root 20667161 Jun  3  2015 /usr/lpp/mmfs/bin/mmfsd
[root@r6181-d5-us01 ~]#

```





واجهة نظام التشغيل للمستخدم GUI -

- واجهة استعارة **سطح المكتب** سهلة الاستخدام
 - عادةً ما يكون الماوس ولوحة المفاتيح والشاشة
 - تمثل **الأيقونات** الملفات والبرامج والإجراءات وما إلى ذلك
 - تؤدي أزرار الماوس المتنوعة فوق الكائنات في الواجهة إلى حدوث إجراءات مختلفة (توفير المعلومات، والخيارات، وتنفيذ الوظيفة، وفتح الدليل (المعروف باسم **المجلد**))
- تم اختراعه في مركز زيروكس بارك
- تتضمن العديد من الأنظمة الآن كل من واجهات GUI و CLI
 - Microsoft Windows عبارة عن واجهة مستخدم رسومية مع غلاف "أمر" CLI
 - نظام التشغيل Apple Mac OS X هو واجهة مستخدم رسومية "Aqua" مع نواة UNIX أسفلها وأغلفة متوفرة
 - يحتوي نظاما Linux و Unix على واجهة سطر أوامر مع واجهات رسومية اختيارية، (CDE، (KDE، GNOME)





واجهات شاشات اللمس

تتطلب أجهزة الشاشة التي تعمل باللمس واجهات جديدة

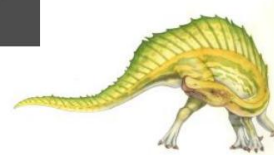
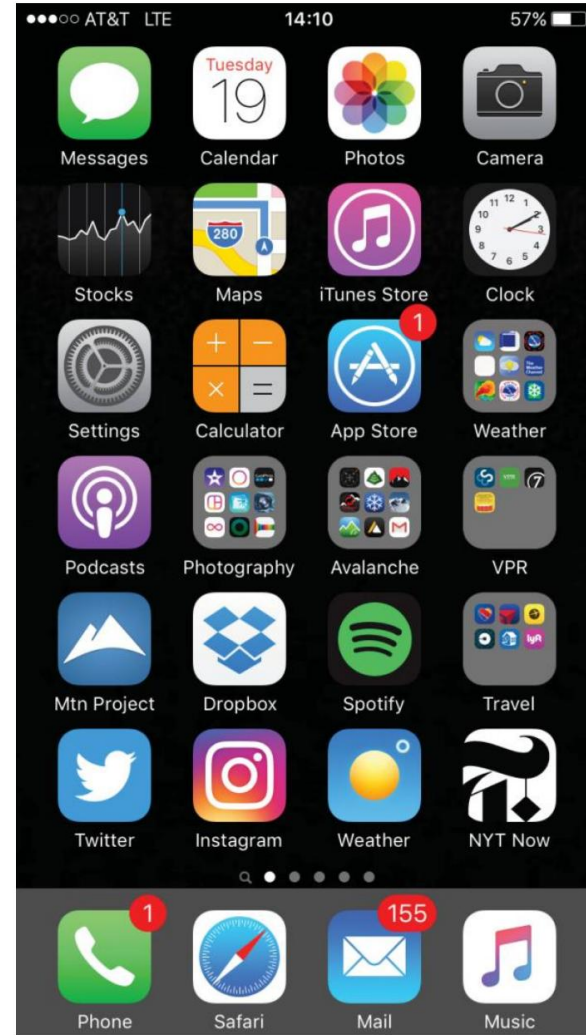
الماوس غير ممكن أو غير مرغوب فيه

ل الإجراءات والاختيار بناءً على

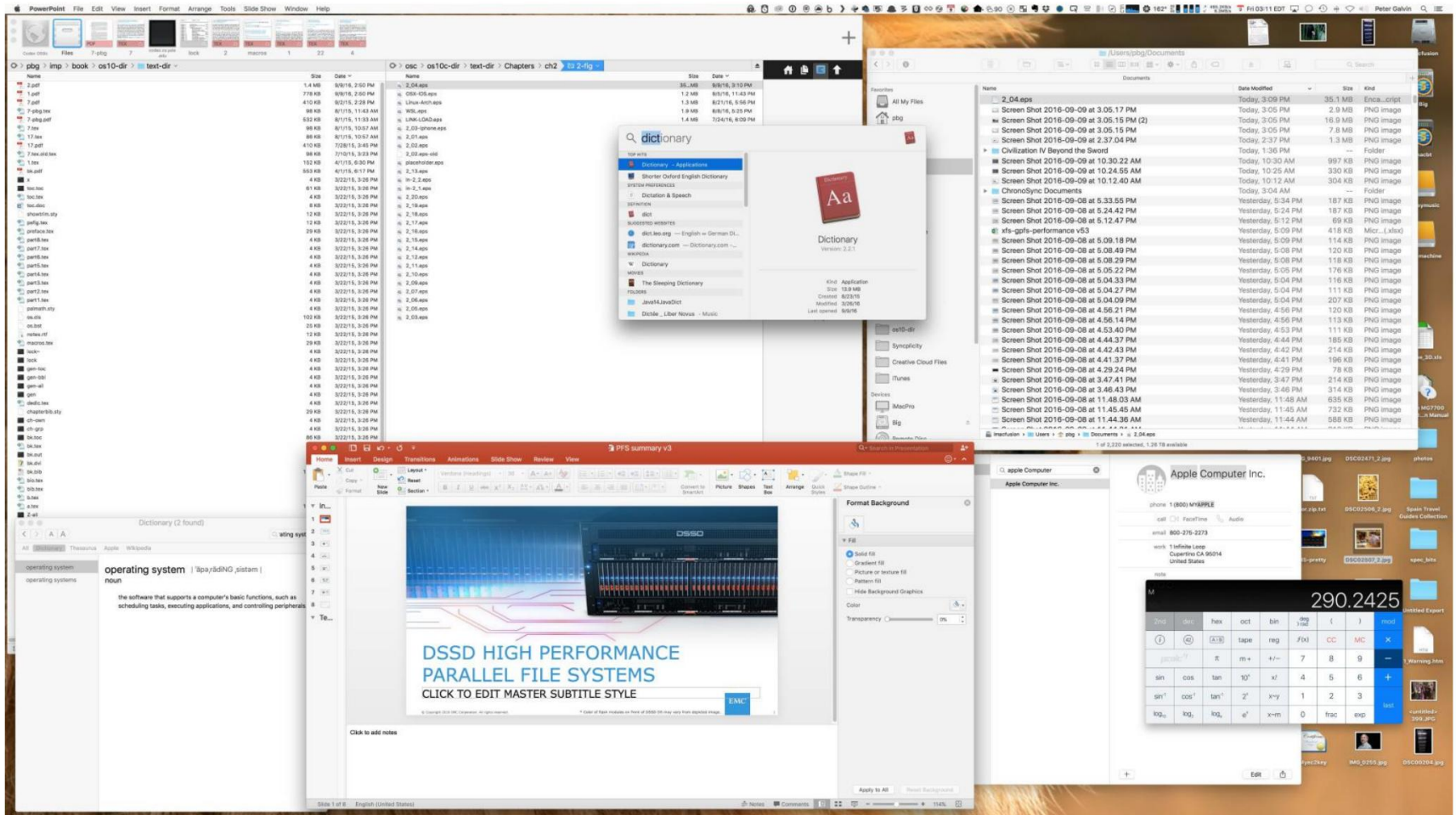
الإيماءات

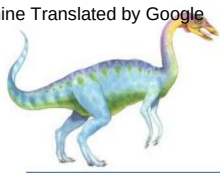
الوحة مفاتيح افتراضية لإدخال النص

الأوامر الصوتية.



واجهة المستخدم الرسومية لنظام التشغيل Mac OS X





مكالمات النظام

- واجهة برمجة للخدمات التي يقدمها نظام التشغيل
- مكتوبة عادة بلغة عالية المستوى (C) أو (C++)
- يتم الوصول إليها في الغالب عن طريق البرامج عبر مستوى عالٍ واجهة برمجة التطبيقات (API) بدلاً من استخدام استدعاء النظام المباشر
- ثلاث واجهات برمجة التطبيقات الأكثر شيوعًا هي Win32 API لنظام التشغيل Windows، واجهة برمجة تطبيقات POSIX للأنظمة المستندة إلى POSIX (بما في ذلك جميع إصدارات Linux وUNIX وOS X تقريبًا)، وواجهة برمجة تطبيقات Java للآلة الافتراضية (JVM) Java

لاحظ أن أسماء استدعاءات النظام المستخدمة في هذا النص عامة

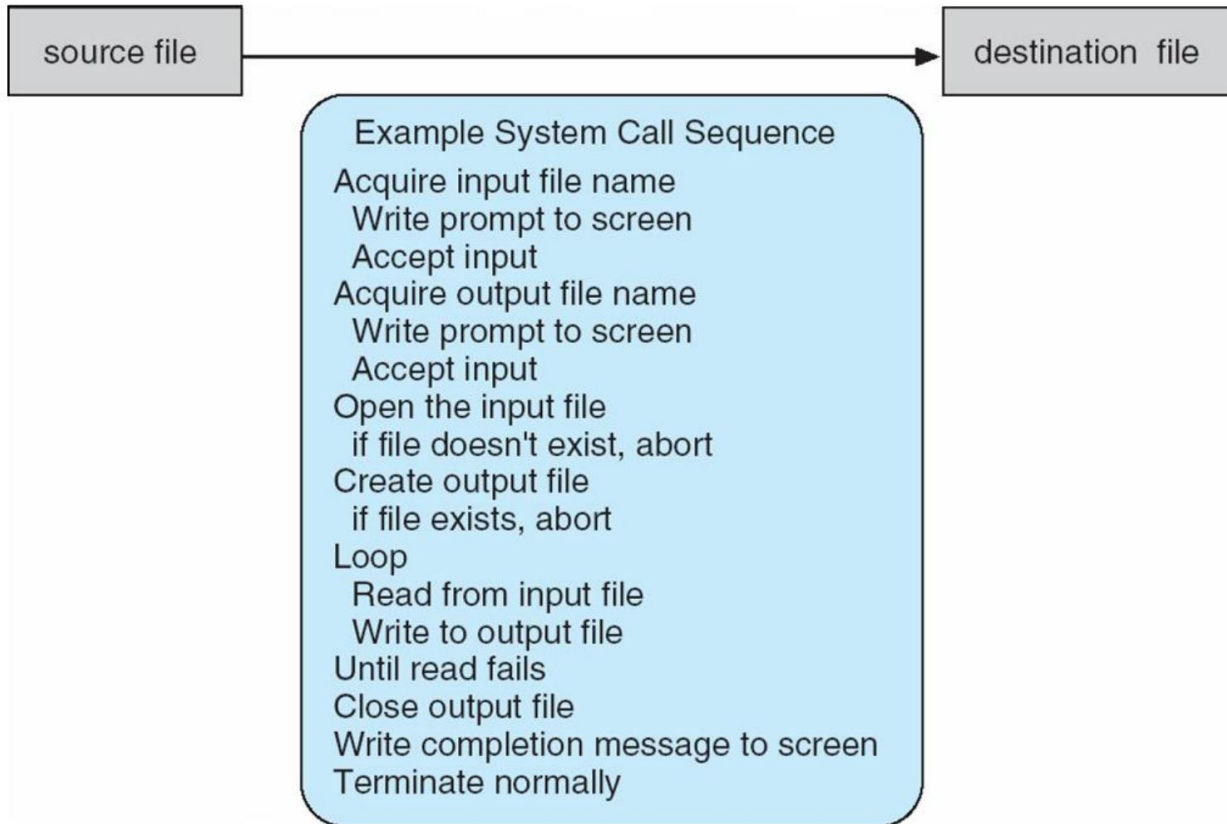




مثال على نداءات النظام



تسلسل استدعاء النظام لنسخ محتويات ملف واحد إلى ملف آخر





مثال على واجهة برمجة التطبيقات القياسية

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

#include <unistd.h>		
ssize_t	read(int fd, void *buf, size_t count)	
return	function	parameters
value	name	

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.





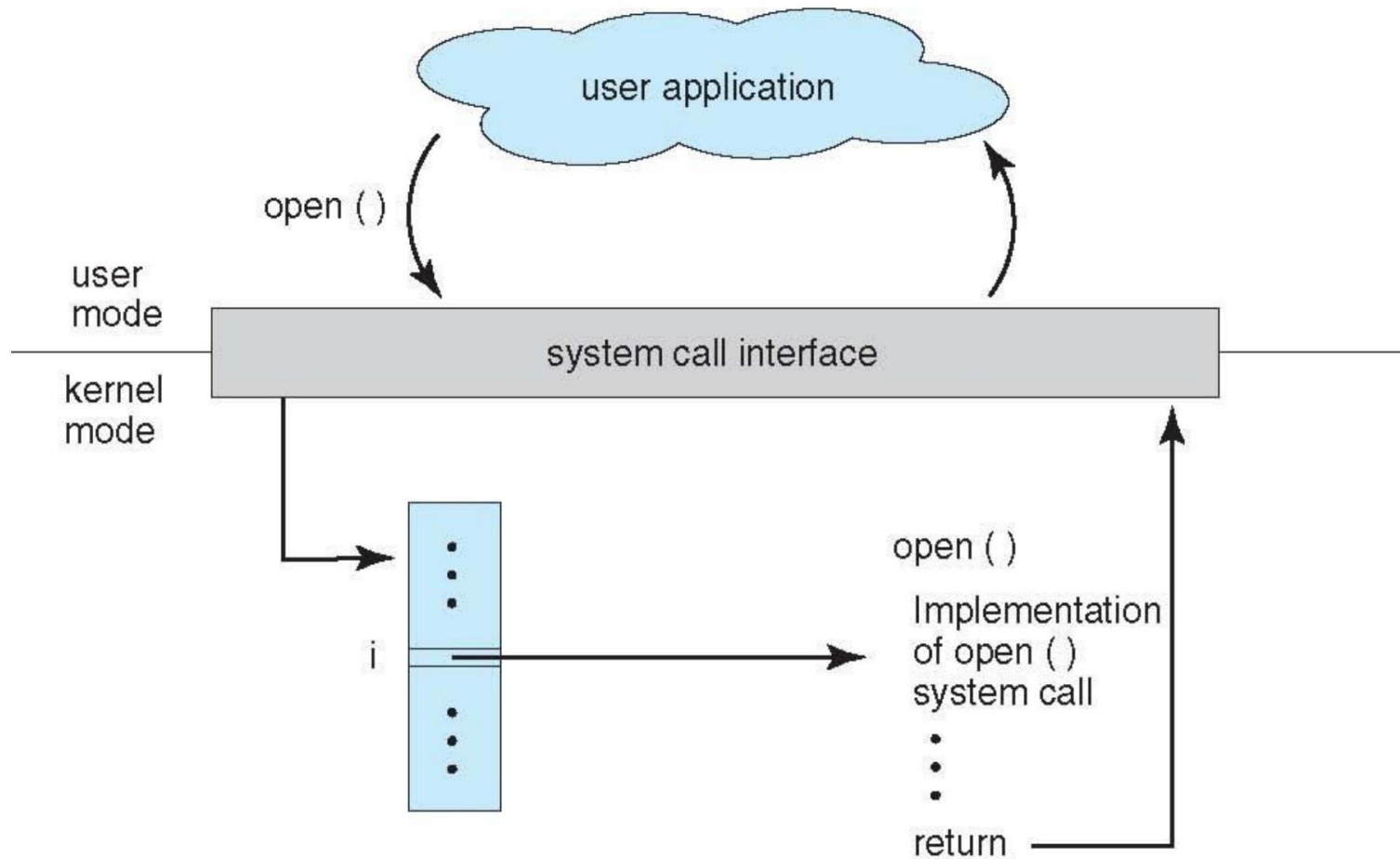
تنفيذ نداء النظام

- عادةً، رقم مرتبط بكل مكانة نظام
- تحافظ واجهة استدعاء النظام على جدول مفهرس وفقاً لهذه الأرقام
- تستدعي واجهة استدعاء النظام استدعاء النظام المقصود في نواة نظام التشغيل وتعيد حالة استدعاء النظام وأي قيم إرجاع
- لا يحتاج المتصل إلى معرفة أي شيء عن كيفية تنفيذ نداء النظام
- كل ما يحتاجه هو الامتثال لواجهة برمجة التطبيقات وفهم ما سيفعله نظام التشغيل كنتيجة للمكانة
- معظم تفاصيل واجهة نظام التشغيل مخفية عن المبرمج بواسطة واجهة برمجة التطبيقات
 - يتم إدارتها بواسطة مكتبة دعم وقت التشغيل (مجموعة من الوظائف المضمنة في المكتبات المضمنة مع المترجم)





API - نداء النظام - علاقة نظام التشغيل



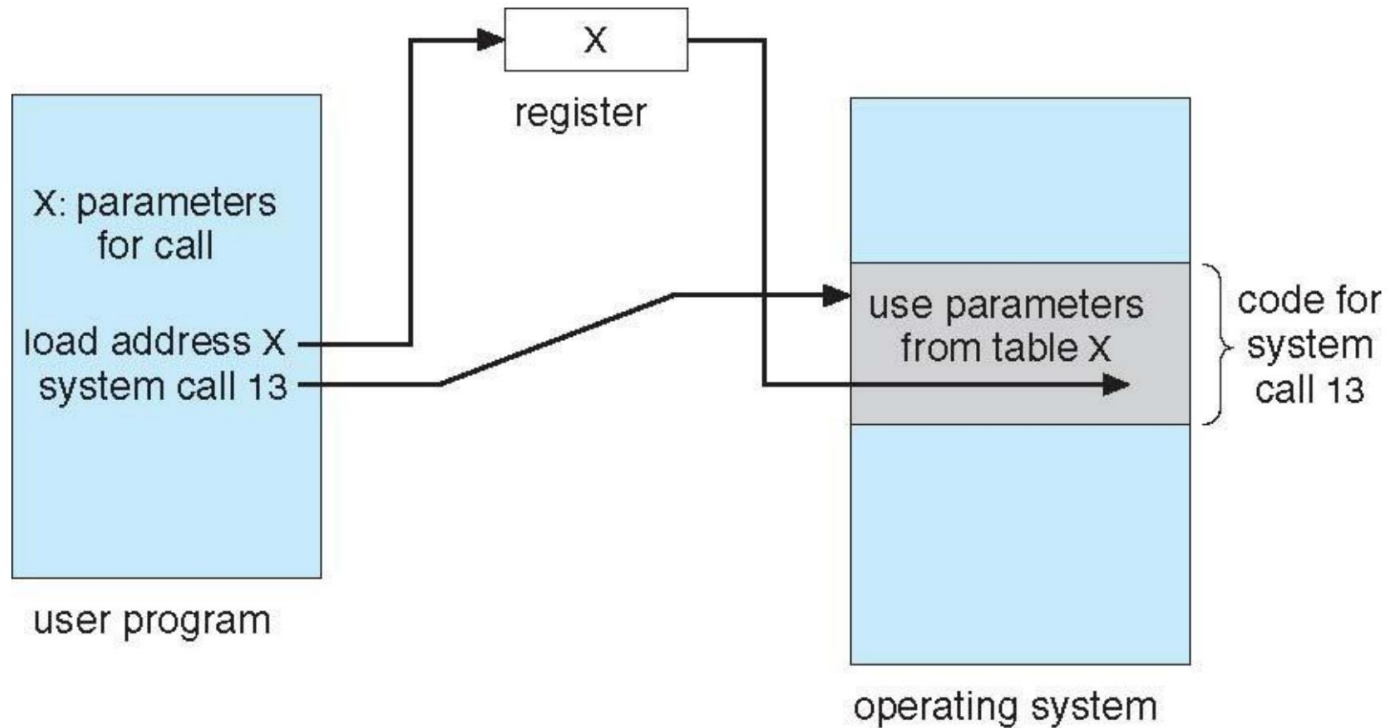
تمرير معلمات نداء النظام

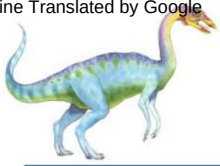
- في كثير من الأحيان، هناك حاجة إلى مزيد من المعلومات أكثر من مجرد هوية نداء النظام المطلوب
- يختلف النوع الدقيق وكمية المعلومات وفقًا لنظام التشغيل والمكالمات
- ثلاث طرق عامة تستخدم لتمرير المعلومات إلى نظام التشغيل
- أبسطها: تمرير المعلومات في السجلات
- في بعض الحالات، قد يكون هناك معلومات أكثر من السجلات
- المعلومات المخزنة في كتلة أو جدول، في الذاكرة، وعنوان الكتلة التي تم تمريرها كمعلمة في سجل □ هذا النهج الذي اتبعته لينكس و Solaris
- المعلومات التي تم وضعها أو دفعها إلى المكدر بواسطة البرنامج وإخراجها من المكدر بواسطة نظام التشغيل
- لا تضع طرق الكتلة والمكدر حدًا لعدد أو طول المعلومات التي يتم تمريرها





تمرير المعلومات عبر الجدول





أنواع نداءات النظام



التحكم في العملية



إنشاء عملية، إنهاء عملية



نهاية، إجهاض



تحميل، تنفيذ



الحصول على سمات العملية، وتعيين سمات العملية



انتظر الوقت



حدث الانتظار، حدث الإشارة، تخصيص وتحرير الذاكرة



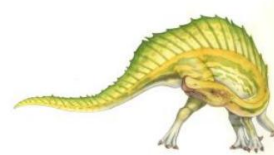
تفريغ الذاكرة في حالة حدوث خطأ



مصحح الأخطاء لتحديد الأخطاء، تنفيذ خطوة واحدة



أقفال لإدارة الوصول إلى البيانات المشتركة بين العمليات

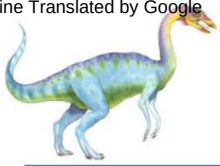




أنواع نداءات النظام

- إدارة الملفات
 - إنشاء ملف، حذف ملف
 - فتح وإغلاق الملف وقراءة وكتابة
 - وإعادة وضع والحصول على سمات
 - الملف وتعيينها
- إدارة الأجهزة
 - طلب الجهاز، إصدار الجهاز
 - قراءة وكتابة وإعادة تحديد موضع الحصول على سمات الجهاز وتعيين
 - سمات الجهاز وربط الأجهزة أو فصلها منطقيًا
 -

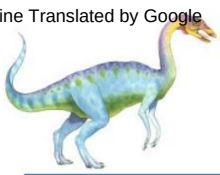




أنواع نداءات النظام (تابع)

- صيانة المعلومات
 - الحصول على الوقت أو التاريخ، وتعيين الوقت أو التاريخ، والحصول
 - على بيانات النظام، وتعيين بيانات النظام، والحصول على سمات العملية
 - أو الملف أو الجهاز وتعيينها
- الاتصالات
 - إنشاء، حذف اتصال الاتصال
 - إرسال واستقبال الرسائل إذا كانت الرسالة تمرر النموذج إلى اسم المضيف أو اسم العملية
- من العميل إلى الخادم
 - نموذج الذاكرة المشتركة ينشئ مناطق الذاكرة ويحصل على إمكانية الوصول إليها
 - معلومات حالة النقل
 - ربط وفصل الأجهزة البعيدة





أنواع نداءات النظام (تابع)



حماية



التحكم في الوصول إلى الموارد



الحصول على الأذونات وتعيينها



السماح بالوصول للمستخدم ورفضه





أمثلة على مكالمات نظام Unix و Windows

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

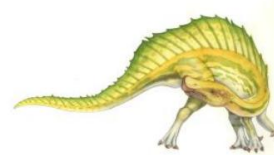
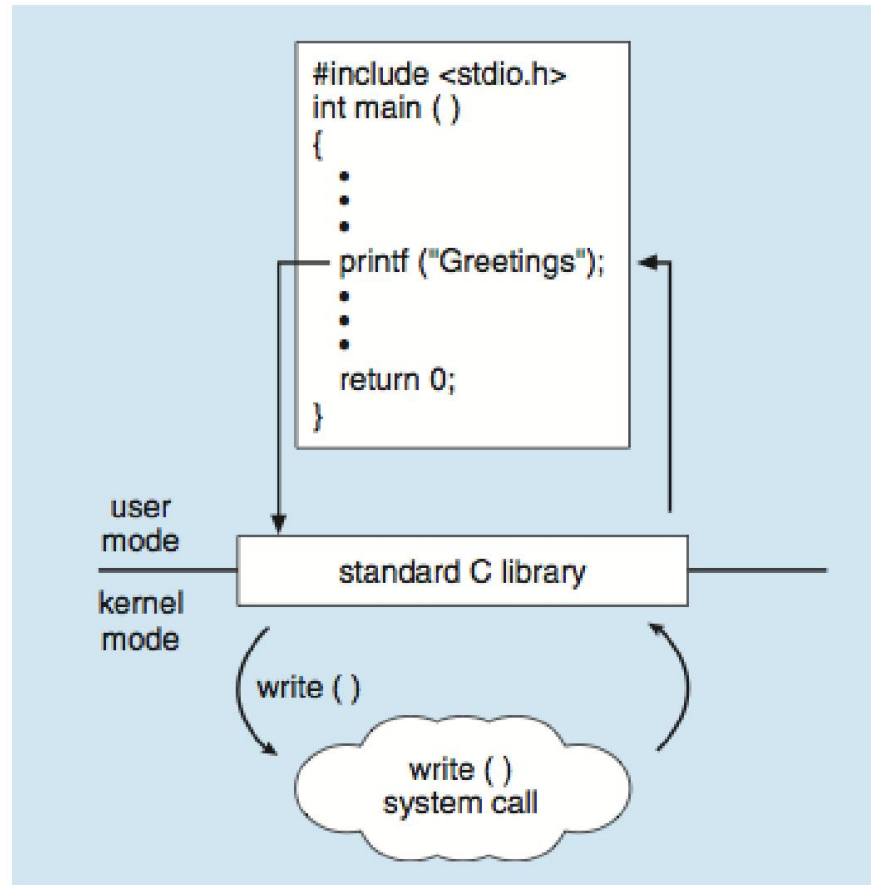




مثال لمكتبة C القياسية



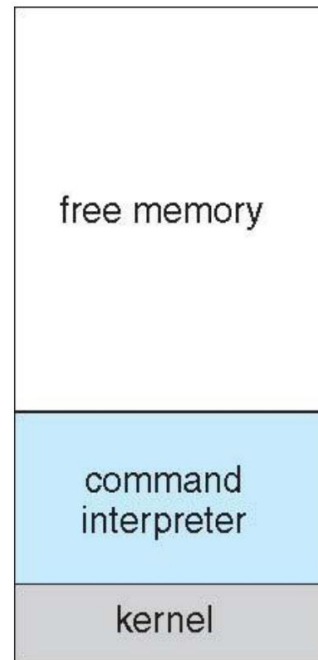
برنامج C يستدعي استدعاء مكتبة `printf()`، والذي يستدعي استدعاء النظام `write()`





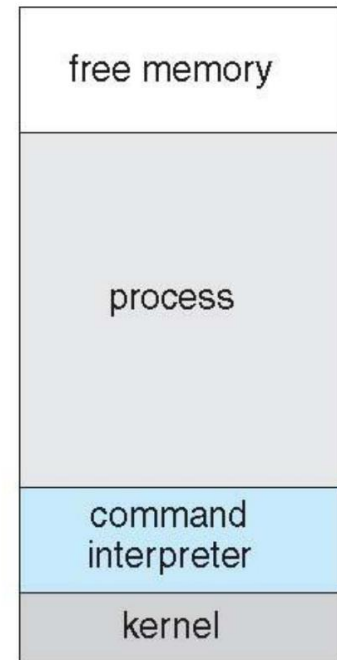
مثال: MS-DOS

- مهمة واحدة
- تم استدعاء Shell عند تشغيل النظام
- طريقة بسيطة لتشغيل البرنامج
- لم يتم إنشاء أي عملية
- مساحة ذاكرة واحدة
- يقوم بتحميل البرنامج في الذاكرة، ويكتب فوق كل شيء باستثناء النواة
- خروج البرنامج -> إعادة تحميل shell



(a)

عند بدء تشغيل النظام



(b)

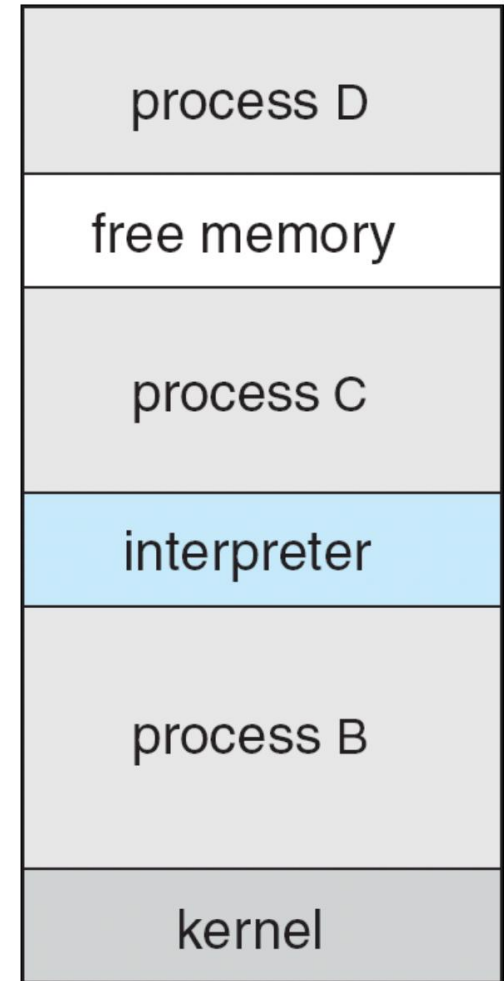
تشغيل برنامج





مثال: FreeBSD

- ❑ متغير يونكس
- ❑ تعدد المهام
- ❑ تسجيل دخول المستخدم -> استدعاء اختيار المستخدم للقشرة
- ❑ يقوم Shell بتنفيذ نداء النظام `fork()` لإنشاء عملية
- ❑ تنفيذ `exec()` لتحميل البرنامج في العملية
- ❑ ينتظر Shell انتهاء العملية أو يستمر باستخدام أوامر المستخدم
- ❑ تخرج العملية مع:
- ❑ الكود - 0 = لا يوجد خطأ
- ❑ الرمز - 0 > رمز الخطأ





برامج النظام

□ تُوفّر برامج النظام بيئةً مُلائمةً لتطوير البرامج وتنفيذها. ويمكن تقسيمها إلى:

□

معالجة الملفات

□

يتم تخزين معلومات الحالة أحياناً في تعديل الملف

□

دعم لغات البرمجة

□

تحميل البرنامج وتنفيذه

□

الاتصالات

□

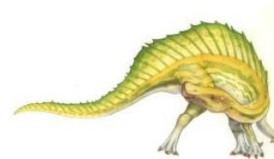
الخدمات الخلفية

□

برامج التطبيقات

□

يتم تحديد وجهة نظر معظم المستخدمين لنظام التشغيل من خلال برامج النظام،
وليس استدعاءات النظام الفعلية





برامج النظام



توفير بيئة ملائمة لتطوير البرنامج وتنفيذه



بعضها عبارة عن واجهات مستخدم لمكالمات النظام؛ والبعض الآخر أكثر تعقيدًا إلى حد كبير



إدارة الملفات - إنشاء الملفات والدلائل وحذفها ونسخها وإعادة تسميتها وطباعتها وتفريغها وإدراجها ومعالجتها بشكل عام



معلومات الحالة



يطلب البعض من النظام معلومات - التاريخ والوقت وكمية الذاكرة المتوفرة ومساحة القرص وعدد المستخدمين



يقدم البعض الآخر معلومات تفصيلية عن الأداء والتسجيل وتصحيح الأخطاء



عادةً، تقوم هذه البرامج بتنسيق الإخراج وطباعته على الجهاز الطرفي أو أجهزة الإخراج الأخرى



تطبق بعض الأنظمة **سجلًا** - يُستخدم لتخزين معلومات التكوين واسترجاعها

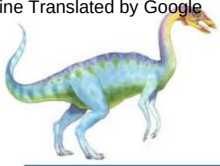




برامج النظام (تابع)

- تعديل الملف
 - محررات النصوص لإنشاء الملفات وتعديلها
 - أوامر خاصة للبحث في محتويات الملفات أو إجراء تحويلات للنص
- دعم لغات البرمجة -يتم توفير المترجمين والمجمعين والمفسرين في بعض الأحيان
- تحميل البرامج وتنفيذها -المحملات المطلقة، والمحملات القابلة للنقل، ومحرري الارتباطات، والمحملات المتراكبة، وأنظمة تصحيح الأخطاء للغة الآلة والمستوى الأعلى
- الاتصالات -توفير الآلية لإنشاء اتصالات افتراضية بين العمليات والمستخدمين وأنظمة الكمبيوتر
- السماح للمستخدمين بإرسال رسائل إلى شاشات بعضهم البعض، وتصفح صفحات الويب، وإرسال رسائل البريد الإلكتروني، وتسجيل الدخول عن بعد، ونقل الملفات من جهاز إلى آخر





برامج النظام (تابع)

- الخدمات الخلفية
 - التشغيل في وقت التمهيد
 - بعضها لبدء تشغيل النظام، ثم إنهائه □ بعضها من تمهيد النظام إلى إيقاف التشغيل
 - توفير مرافق مثل فحص القرص، وجدولة العمليات، وتسجيل الأخطاء، والطباعة
 - تشغيل في سياق المستخدم وليس سياق النواة
 - تُعرف باسم الخدمات والأنظمة الفرعية والشياطين
- برامج التطبيقات
 - لا تتعلق بالنظام
 - تشغيلها من قبل المستخدمين
 - لا يُعتبر عادةً جزءًا من نظام التشغيل
 - يتم تشغيله عن طريق سطر الأوامر، أو النقر بالماوس، أو نقرة الإصبع





تصميم وتنفيذ نظام التشغيل

- تصميم وتنفيذ أنظمة التشغيل ليس "قابلاً للحل"، ولكن بعض الأساليب أثبتت نجاحها
- يمكن أن يختلف الهيكل الداخلي لأنظمة التشغيل المختلفة على نطاق واسع
- ابدأ التصميم بتحديد الأهداف والمواصفات
- تتأثر باختيار الأجهزة ونوع النظام
- أهداف المستخدم وأهداف النظام
- أهداف المستخدم - يجب أن يكون نظام التشغيل سهل الاستخدام وسهل التعلم وموثوقاً وآمناً وسريعاً
- أهداف النظام - يجب أن يكون نظام التشغيل سهل التصميم والتنفيذ والصيانة، بالإضافة إلى كونه مرناً وموثوقاً وخالياً من الأخطاء وفعالاً





تصميم وتنفيذ نظام التشغيل (تابع)



مبدأ مهم للفصل

هل هذا ذلك؟
هل هذا ذلك؟



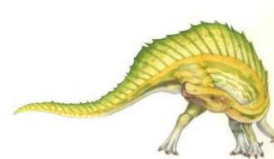
تحدد الآليات كيفية القيام بشيء ما، وتحدد السياسات ما سيتم القيام به

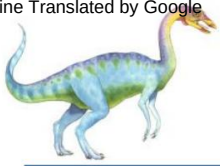


إن فصل السياسة عن الآلية مبدأ مهم للغاية، فهو يسمح بأقصى قدر من المرونة إذا كان من المقرر تغيير قرارات السياسة لاحقًا (مثال - المؤقت)



إن تحديد وتصميم نظام التشغيل يعد مهمة إبداعية للغاية في مجال **هندسة البرمجيات**





تطبيق



الكثير من التنوع



أنظمة التشغيل المبكرة في لغة التجميع



ثم لغات برمجة النظام مثل Algol وLP/1



الآن C، C++



في الواقع عادة ما يكون مزيج من اللغات



أدنى مستويات التجميع



الجسم الرئيسي في C



برامج الأنظمة بلغة C وC++ ولغات البرمجة النصية مثل PERL،

بايثون، نصوص شل



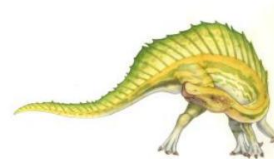
لغة أكثر مستوى أعلى أسهل في [النقل](#) إلى أجهزة أخرى

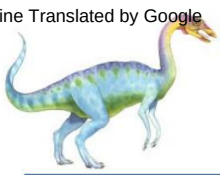


ولكن أبطأ



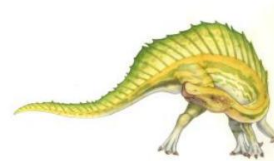
يمكن أن يسمح [المحاكاة](#) لنظام التشغيل بالعمل على أجهزة غير أصلية





بنية نظام التشغيل

- نظام التشغيل للأغراض العامة هو برنامج كبير جدًا
- طرق مختلفة لتنظيم بعضها
 - بنية بسيطة MS-DOS -
 - أكثر تعقيدًا -- يونكس
 - متعدد الطبقات - تجريد
 - نواة دقيقة - ماخ



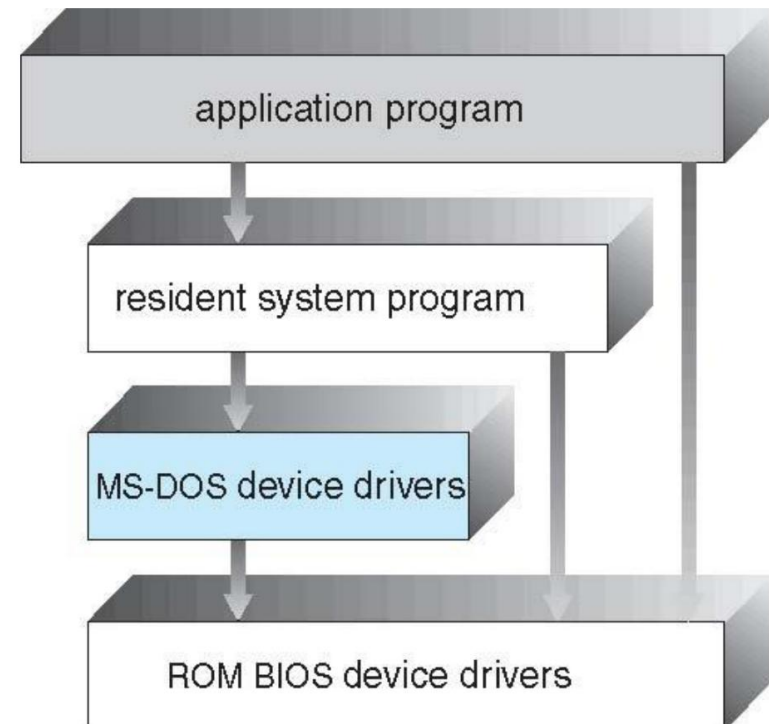


-- بنية بسيطة MS-DOS

□ - MS-DOS مكتوب لتوفير أقصى قدر من الوظائف في أقل مساحة

□ غير مقسمة إلى وحدات

□ على الرغم من أن MS-DOS لديه بعض البنية، إلا أن واجهاته ومستويات وظائفه ليست منفصلة بشكل جيد





البنية غير البسيطة -- يونكس

يوناكس -كان نظام التشغيل يوناكس الأصلي محدودًا بوظائف الأجهزة، وكان هيكله محدودًا. يتكون نظام تشغيل يوناكس من جزأين منفصلين.



برامج النظم



النواة

يتكون من كل شيء أسفل واجهة استدعاء النظام وفوق الأجهزة المادية

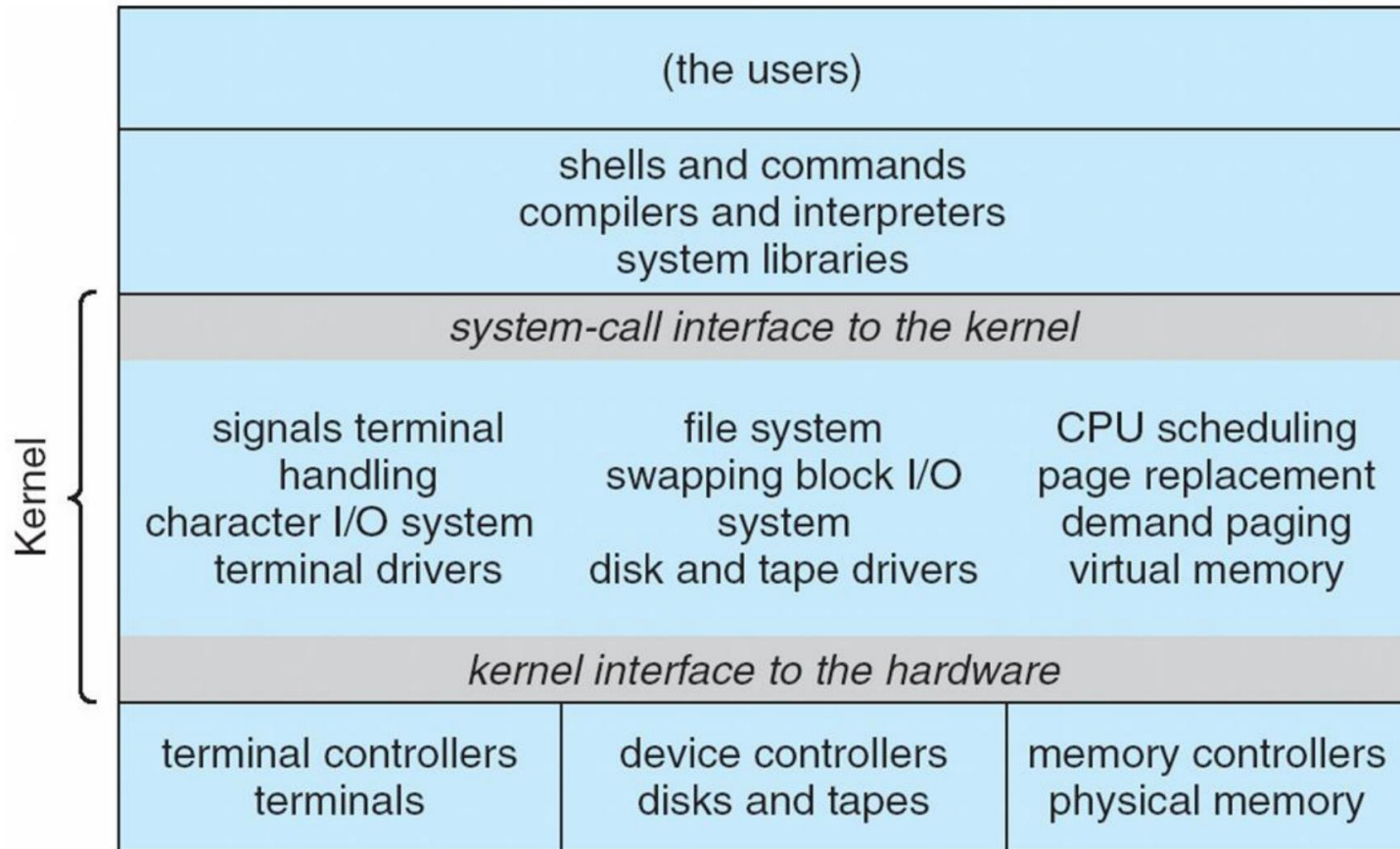
يوفر نظام الملفات وجدولة وحدة المعالجة المركزية والذاكرة الإدارة، ووظائف نظام التشغيل الأخرى؛ عدد كبير من الوظائف لمستوى واحد





بنية نظام يونكس التقليدي

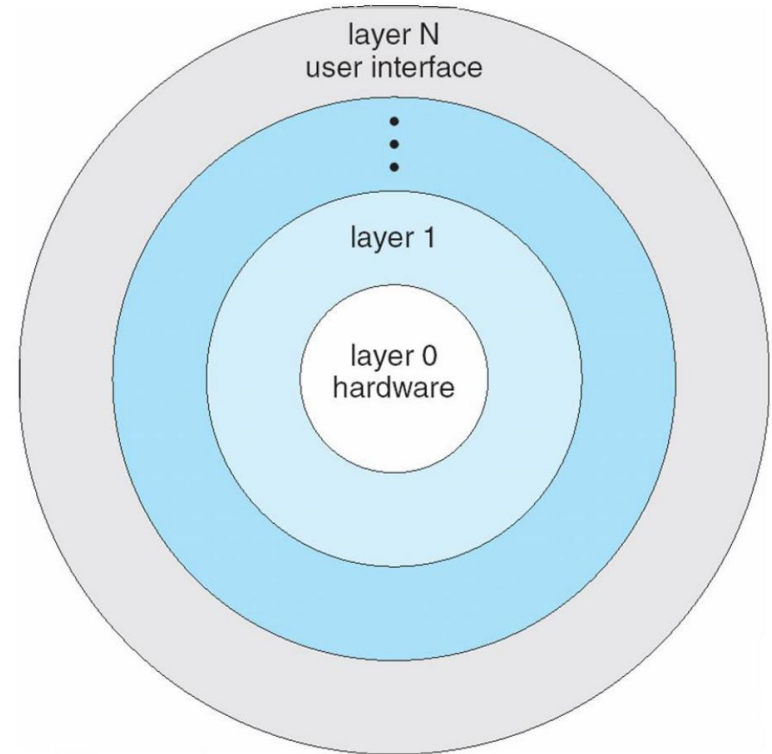
أبعد من البساطة ولكن ليس بشكل كامل





النهج الطبقي

- ينقسم نظام التشغيل إلى عدة طبقات، كل منها مبني فوق طبقات أدنى. الطبقة السفلى (الطبقة 0) هي الأجهزة؛ والطبقة العليا (الطبقة N) هي واجهة المستخدم.
- في حالة الوحدات النمطية، يتم تحديد الطبقات بحيث تستخدم كل منها وظائف (عمليات) وخدمات الطبقات ذات المستوى الأدنى فقط





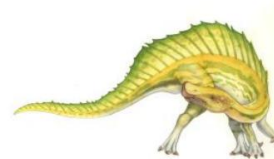
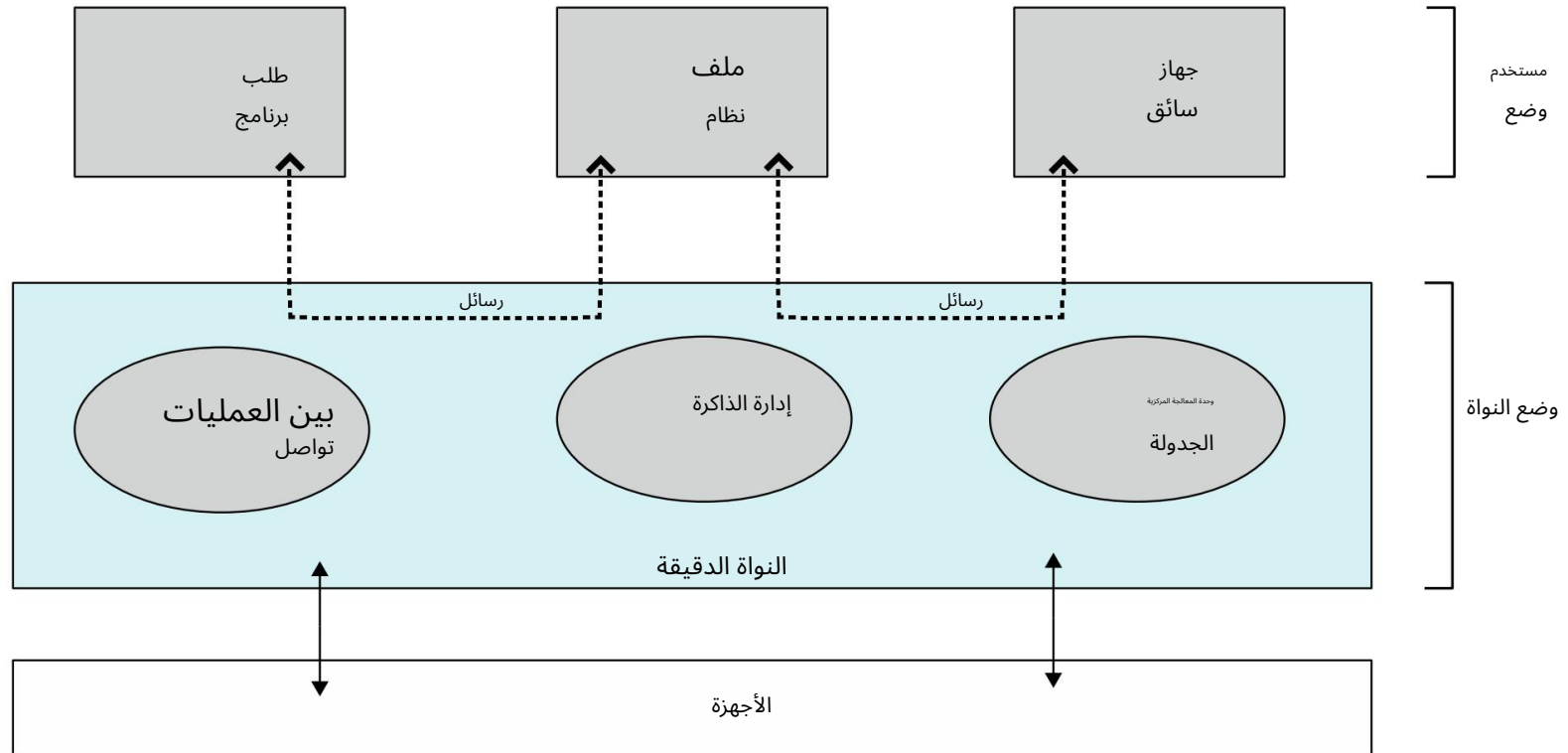
بنية نظام النواة الدقيقة

- ينتقل قدر الإمكان من النواة إلى مساحة المستخدم
- مثال **ماخ** على **النواة الدقيقة**
- نواة **Mac OS X (Darwin)** تعتمد جزئيًا على **Mach**
- يتم إجراء الاتصال بين وحدات المستخدم باستخدام **تمرير الرسائل**
- فوائد:
- من الأسهل توسيع النواة الدقيقة
- أصبح من الأسهل نقل نظام التشغيل إلى البنيات المعمارية الجديدة
- أكثر موثوقية (يتم تشغيل كمية أقل من التعليمات البرمجية في وضع (kernel
- أكثر أمانًا
- الأضرار:
- تكلفة الأداء للاتصال بين مساحة المستخدم ومساحة النواة





بنية نظام النواة الدقيقة





الوحدات النمطية



تطبق العديد من أنظمة التشغيل الحديثة وحدات النواة القابلة للتحميل



يستخدم نهجًا موجّهًا نحو الكائنات



كل مكون أساسي منفصل



كل واحد يتحدث مع الآخرين عبر واجهات معروفة



يمكن تحميل كل منها حسب الحاجة داخل النواة



بشكل عام، مشابهة للطبقات ولكن مع مرونة أكبر

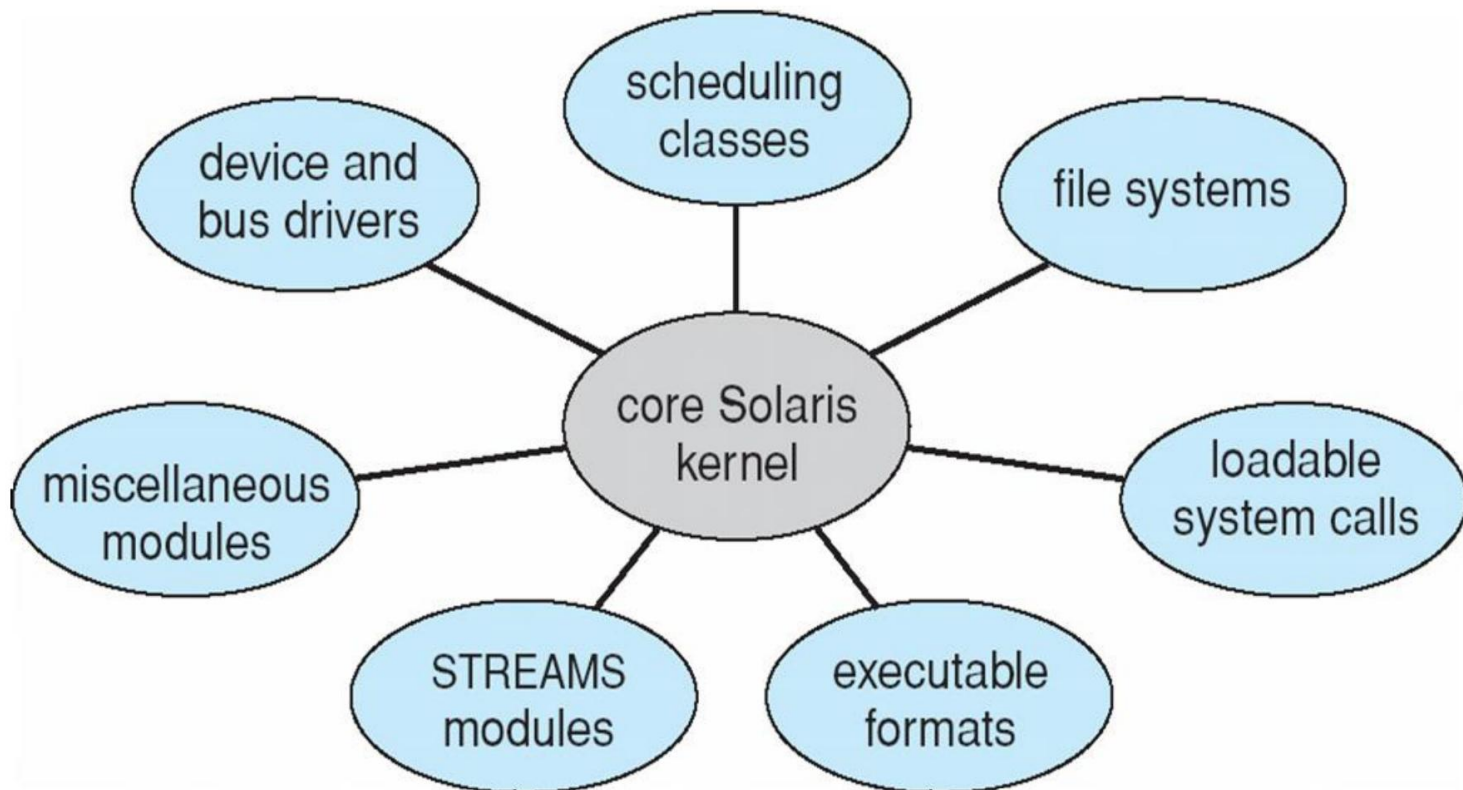


لينكس، سولاريس، الخ





نهج سولاريس المعياري

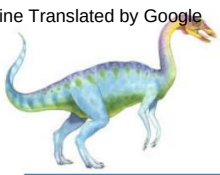




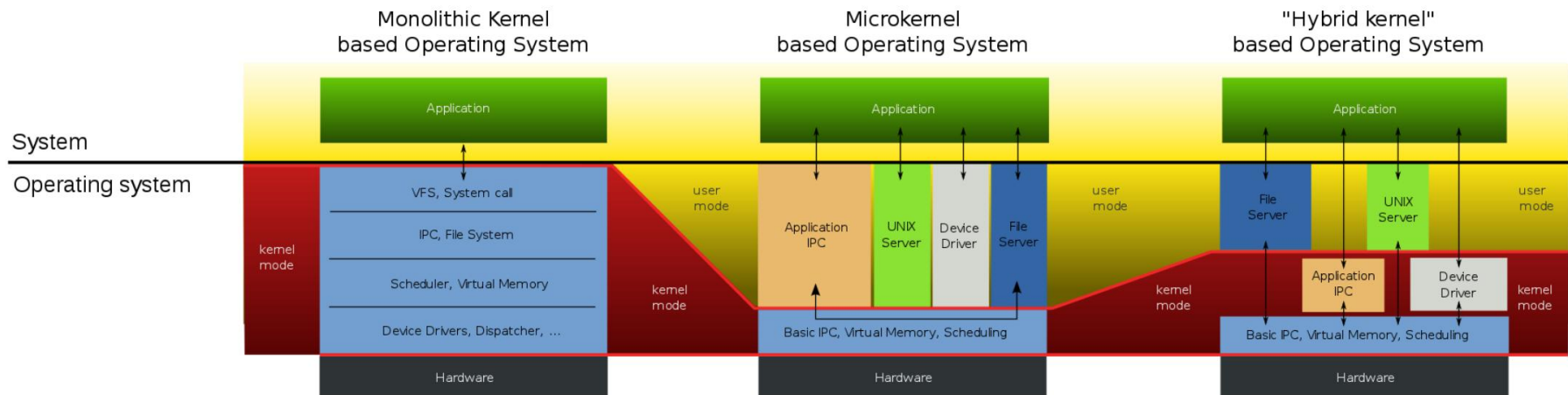
الأنظمة الهجينة

- في الواقع، لا تعد معظم أنظمة التشغيل الحديثة نموذجًا واحدًا خالصًا
- يجمع النظام الهجين بين عدة مناهج لتلبية احتياجات الأداء والأمان وسهولة الاستخدام
- أنوية Solaris وLinux في مساحة عنوان النواة، وبالتالي فهي متجانسة، بالإضافة إلى كونها معيارية للتحميل الديناميكي للوظائف
- نظام Windows متجانس في الغالب، بالإضافة إلى نواة صغيرة لأنظمة فرعية مختلفة الشخصيات
- نظام التشغيل Apple Mac OS X الهجين متعدد الطبقات مع واجهة مستخدم Aqua بالإضافة إلى Cocoa بيئة البرمجة
- فيما يلي نواة تتكون من نواة Mach الدقيقة وأجزاء BSD Unix، بالإضافة إلى مجموعة I/O ووحدات قابلة للتحميل ديناميكيًا (تسمى ملحقات النواة)





بنية نظام التشغيل

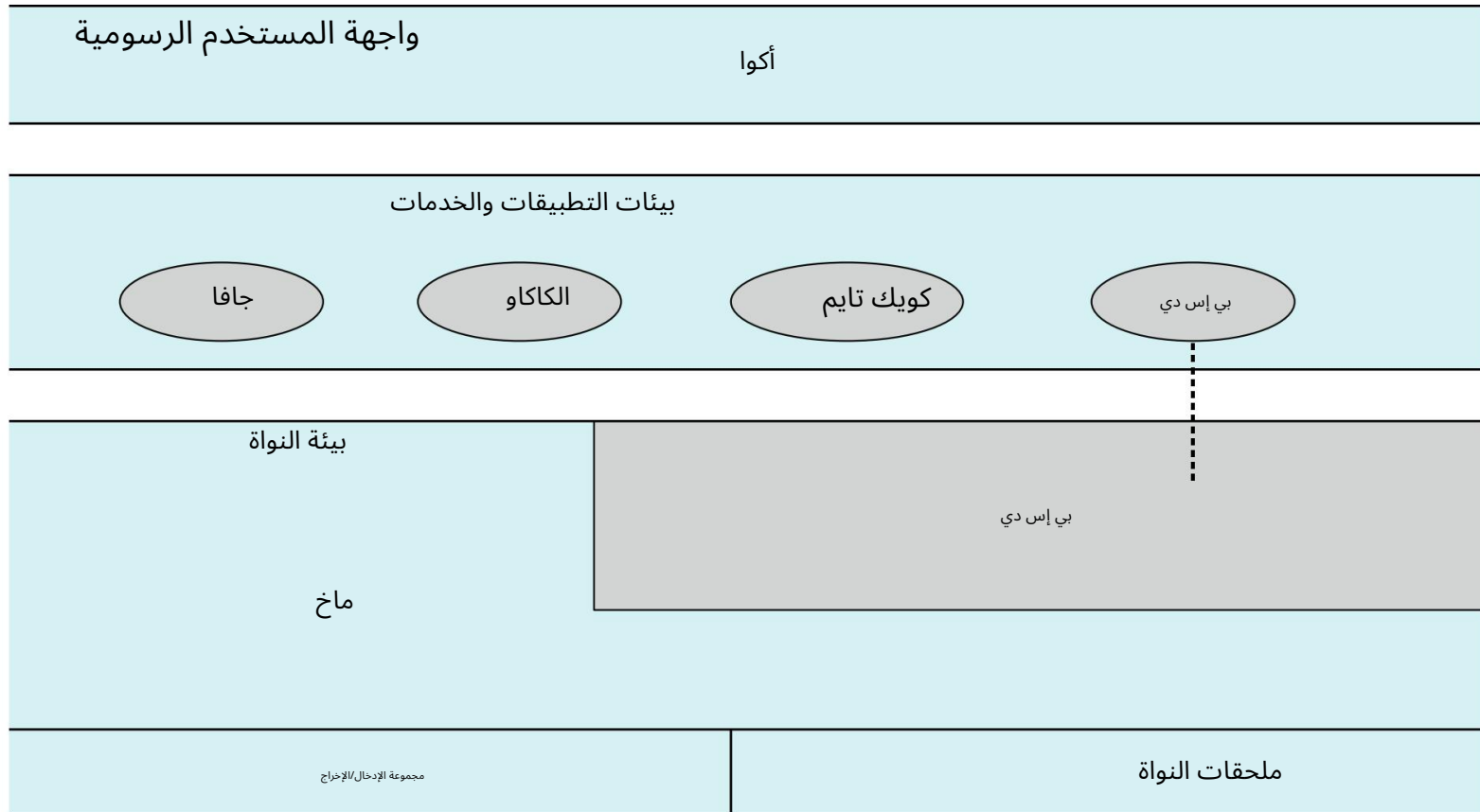


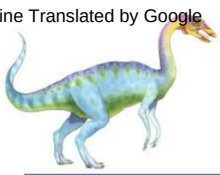
المصدر: https://en.wikipedia.org/wiki/Monolithic_kernel





بنية نظام التشغيل Mac OS X





اي او اس



نظام تشغيل Apple للجوال



تم تصميمه على نظام التشغيل Mac OS X، مع إضافة وظائف



لا يقوم بتشغيل تطبيقات OS X بشكل أصلي

يعمل أيضًا على بنية وحدة المعالجة المركزية المختلفة

(ARM) مقابل (Intel)



واجهة برمجة تطبيقات Objective-C Cocoa Touch لتطوير التطبيقات



طبقة خدمات الوسائط للرسومات والصوت والفيديو



توفر الخدمات الأساسية الحوسبة السحابية وقواعد البيانات



نظام التشغيل الأساسي، استنادًا إلى نواة Mac OS X

Cocoa Touch

Media Services

Core Services

Core OS

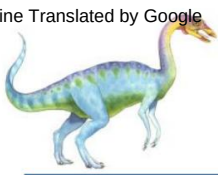




أندرويد

- تم تطويره بواسطة Open Handset Alliance (معظمها من Google)
- مفتوح المصدر
- مجموعة مماثلة لنظام IOS
- مبني على نواة لينكس ولكن تم تعديله
- يوفر إدارة العمليات والذاكرة وبرامج تشغيل الأجهزة
- إضافة إدارة الطاقة
- تتضمن بيئة التشغيل مجموعة أساسية من المكتبات وآلة Dalvik الافتراضية
- التطبيقات التي تم تطويرها بلغة Java بالإضافة إلى واجهة برمجة تطبيقات Android
- ملفات فئة Java تم تجميعها إلى بايت كود Java ثم ترجمتها إلى ملف قابل للتنفيذ ثم تشغيلها في VM Dalvik
- تتضمن المكتبات أطر عمل لمتصفح الويب (webkit) وقاعدة البيانات (SQLite)، الوسائط المتعددة، libC أصغر





هندسة أندرويد

التطبيقات

إطار عمل التطبيق

المكتبات

SQLite

أوبن جي إل

سطح
مدير

وسائط
نطاق

ويب كيت

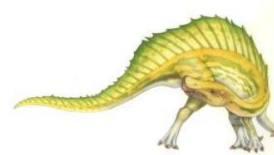
مكتبة سي

وقت تشغيل أندرويد

المكتبات الأساسية

دالفيك
آلة افتراضية

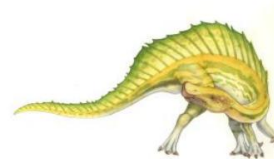
نواة لينكس





تصحيح أخطاء نظام التشغيل

- التصحيح هو العثور على الأخطاء وإصلاحها أو الخلل
 - يقوم نظام التشغيل بإنشاء ملفات سجل تحتوي على معلومات الخطأ
 - قد يؤدي فشل التطبيق إلى إنشاء ملف تفريغ أساسي يلتقط ذاكرة العملية
 - قد يؤدي فشل نظام التشغيل إلى إنشاء ملف تفريغ الأعطال الذي يحتوي على ذاكرة النواة
 - بالإضافة إلى الأعطال، يمكن لضبط الأداء تحسين أداء النظام
 - مؤشرات الأداء الرئيسية للتطبيق
 - التنميط هو أخذ عينات دورية من مؤشر التعليمات للبحث عن الاتجاهات الإحصائية
- قانون كيرنيغان: "تصحيح الأخطاء أصعب بمرتين من كتابة الكود أصلاً . لذلك، إذا كتبت الكود بأقصى قدر ممكن من الذكاء، فأنت ، بحكم التعريف، لست ذكيًا بما يكفي لتصحيحه."





ضبط الأداء



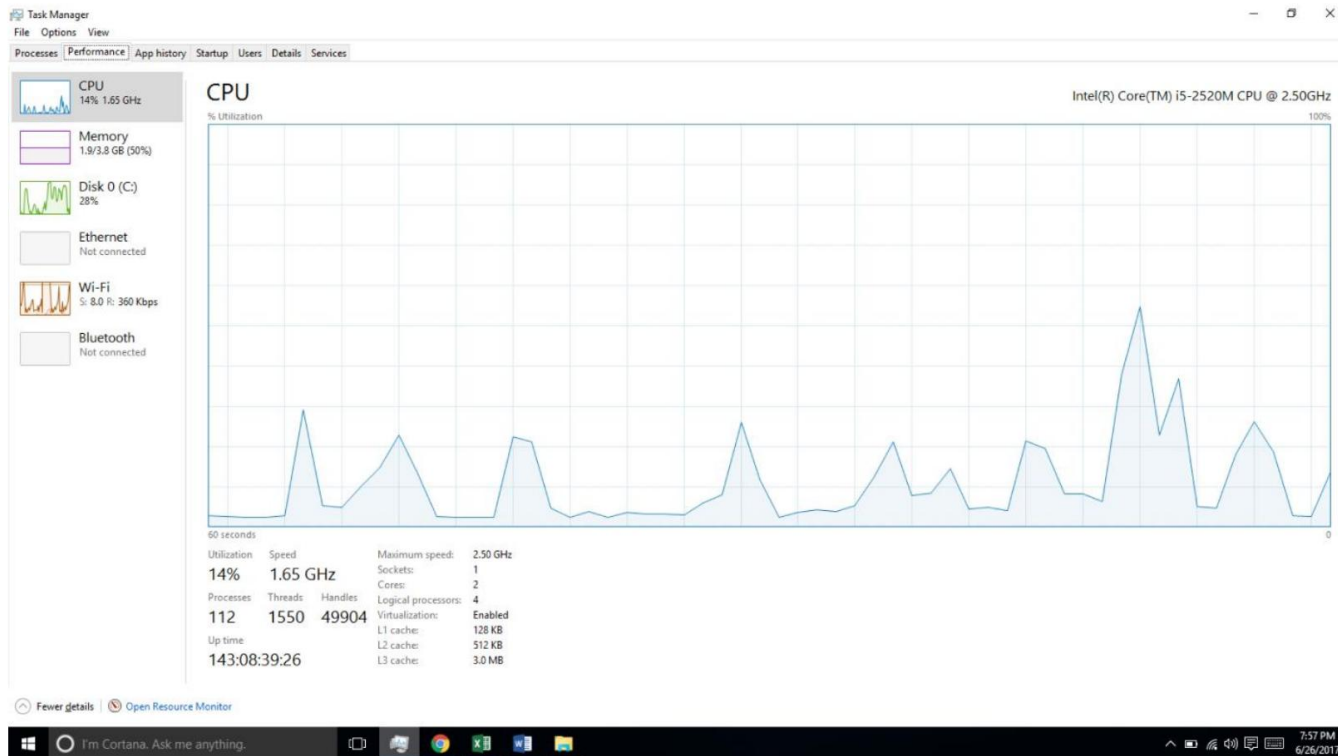
تحسين الأداء عن طريق إزالة الاختناقات



يجب أن يوفر نظام التشغيل وسائل حساب وعرض مقاييس سلوك النظام



على سبيل المثال، برنامج "top" أو "إدارة مهام Windows"





دي تريس



أداة DTrace في Solaris، FreeBSD و OS X caM يسمحان بالقياس المباشر على أنظمة الإنتاج

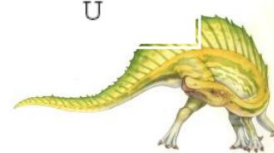


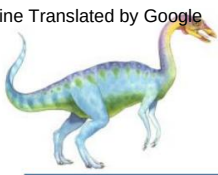
يتم إطلاق [المجسات](#) عند تنفيذ التعليمات البرمجية داخل [مزود الخدمة](#)، والتقاط بيانات الحالة وإرسالها إلى [مستهلكي](#) هذه المجسات



مثال على ما يلي
نقل نداء نظام XEventsQueued من مكتبة libc إلى kernel والعكس

```
# ./all.d 'pgrep xclock' XEventsQueued
dtrace: script './all.d' matched 52377 probes
CPU FUNCTION
0 -> XEventsQueued U
0 -> _XEventsQueued U
0 -> _X11TransBytesReadable U
0 <- _X11TransBytesReadable U
0 -> _X11TransSocketBytesReadable U
0 <- _X11TransSocketBytesreadable U
0 -> ioctl U
0 -> ioctl K
0 -> getf K
0 -> set_active_fd K
0 <- set_active_fd K
0 <- getf K
0 -> get_udatamodel K
0 <- get_udatamodel K
...
0 -> releasef K
0 -> clear_active_fd K
0 <- clear_active_fd K
0 -> cv_broadcast K
0 <- cv_broadcast K
0 <- releasef K
0 <- ioctl K
0 <- ioctl U
0 <- _XEventsQueued U
0 <- XEventsQueued U
```





دي تريس (تابع)

- كود DTrace لتسجيل مقدار الوقت الذي تكون فيه كل عملية ذات معرف المستخدم 101 في وضع التشغيل (على وحدة المعالجة المركزية) بالنانوثانية

```

sched::on-cpu
uid == 101
{
    self->ts = timestamp;
}

sched::off-cpu
self->ts
{
    @time[execname] = sum(timestamp - self->ts);
    self->ts = 0;
}

```

```

# dtrace -s sched.d
dtrace: script 'sched.d' matched 6 probes
^C

```

gnome-settings-d	142354
gnome-vfs-daemon	158243
dsdm	189804
wnck-applet	200030
gnome-panel	277864
clock-applet	374916
mapping-daemon	385475
xscreensaver	514177
metacity	539281
Xorg	2579646
gnome-terminal	5007269
mixer_applet2	7388447
java	10769137

Figure 2.21 Output of the D code.





إنشاء نظام التشغيل

تم تصميم **أنظمة** التشغيل للعمل على أي فئة من الأجهزة؛ يجب تكوين النظام لكل موقع كمبيوتر محدد

يُحصل برنامج **SYSGEN** على معلومات تتعلق بـ

تكوين نظام الأجهزة

يستخدم لبناء نواة مجمعة خاصة بالنظام أو نظام
مُضبوط

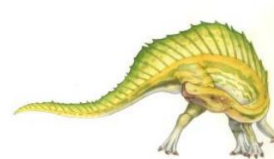
هل يمكن أن يكون الكود العام أكثر كفاءة من نواة عامة واحدة؟

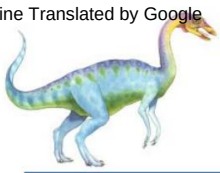




تمهيد النظام

- عند تهيئة الطاقة على النظام، يبدأ التنفيذ في موقع ذاكرة ثابت
- تستخدم ذاكرة القراءة فقط (ROM) الخاصة بالبرامج الثابتة لحفظ رمز التمهيد الأولي
- يجب أن يكون نظام التشغيل متاحًا للأجهزة حتى تتمكن من تشغيله
- قطعة صغيرة من الكود - **محمل التمهيد**، مخزنة في **ROM** أو **EEPROM** لتحديد موقع النواة، ثم يقوم بتحميلها في الذاكرة، ثم يبدأ تشغيلها
- في بعض الأحيان تكون العملية مكونة من خطوتين حيث يتم تحميل **كتلة التمهيد** في موقع ثابت بواسطة كود ROM، والذي يقوم بتحميل محمل التمهيد من القرص
- يسمح برنامج التحميل المشترك **GRUB** باختيار النواة من أقراص متعددة وإصدارات وخيارات النواة
- يتم تحميل النواة ثم يتم **تشغيل** النظام





تصميم وتنفيذ نظام التشغيل

الواجب المنزلي رقم 3



ما هو VxWorks؟ اذكر وصفًا موجزًا وتاريخًا.



ما هي العلاقة بين VxWorks والأنظمة المضمنة؟



ما هي العلاقة بين VxWorks وإنترنت الأشياء (IoT)؟



نهاية الفصل الثاني

